Implementing Projections and Filters

In this train, we'll cover how to apply projections and filters on data in Spark, streamlining data for specific tasks and improving performance.

Learning Objectives

By the end of this train, you should be able to:

- Implement projections and filters on data in Spark.

What is a Projection?

A **projection** in a relational sense is a way to return only the rows matching specific conditions.

A **filter** is used to specify the criteria used for determining which rows to return.

In PySpark, projections are done using the `select()` method, where filters are expressed using the `filter()` or `where()` methods, which are both equivalent.

Implementing a Projection on Data

Let's implement a projection by selecting all entries, in our data, which occur within our data after 2019.

To do this, we're going to use a couple of functions from the `datetime` module that you hopefully should be familiar with. Here we'll construct a `date` object, which can be used to filter `timestamp` data in Spark.
Note that Spark also allows filtering with strings, for example, using `> '2020-01'` to return all dates greater than Jan-01-2020, but it is generally suggested to give the correct type to avoid possible data type mismatch errors between PySpark and Spark.

```
from datetime import date
filter_date = date(2020, 1, 1)
```

```
df.where(F.col('Timestamp') > filter_date).show(5, True)  # <-- Return all
entries later than Jan 01, 2020.
```

You should get an output matching the following

```
+-------------------+-------+-------+-------+-------+-----------+-------------
---+--------------+----+
|          Timestamp|   Open|   High|    Low|
Close|Volume_(BTC)|Volume_(Currency)|Weighted_Price|Year|
+-------------------+-------+-------+-------+-------+-----------+-------------
---+--------------+----+
|2020-01-01 00:01:00|7161.51|7161.51|7155.09| 7161.2|   3.7769244|
27047.305|      7161.198|2020|
|2020-01-01 00:02:00|7158.82|7158.82|7158.82|7158.82|  0.02927792|
209.59535|       7158.82|2020|
|2020-01-01 00:03:00|7158.82|7158.82| 7156.9| 7156.9|  0.06581935|
471.1561|      7158.322|2020|
|2020-01-01 00:04:00| 7158.5| 7158.5|7154.97| 7157.2|  0.97138673|
6950.501|      7155.236|2020|
|2020-01-01 00:05:00|7156.52|7159.51| 7150.1| 7158.5|  0.88693166|
6342.851|      7151.454|2020|
+-------------------+-------+-------+-------+-------+-----------+-------------
---+--------------+----+
only showing top 5 rows
```

We've just filtered based on the row values found within a column, with the resulting DataFrame having its full complement of fields. Often, however, we'd like to constrain the columns we return as part of the projection as well.

To do this we make use of the `select()` method, which allows us to pick a subset of the fields from the target DataFrame:

```
df.select('Timestamp', 'Open', 'High', 'Low').show(5, False)
```

You should get something similar to:

```
+-------------------+----+----+----+
|Timestamp          |Open|High|Low |
+-------------------+----+----+----+
|2011-12-31 07:52:00|4.39|4.39|4.39|
|2011-12-31 07:53:00|NaN |NaN |NaN |
|2011-12-31 07:54:00|NaN |NaN |NaN |
|2011-12-31 07:55:00|NaN |NaN |NaN |
|2011-12-31 07:56:00|NaN |NaN |NaN |
+-------------------+----+----+----+
only showing top 5 rows
```

You can chain this method together with your previously built filter to produce a concise projection:

```
df.select('Timestamp', 'Open', 'High', 'Low').where(F.col('Timestamp') >
filter_date).show(5, False)
```

Which should return

```
+-------------------+-------+-------+-------+
|Timestamp          |Open   |High   |Low    |
+-------------------+-------+-------+-------+
|2020-01-01 00:01:00|7161.51|7161.51|7155.09|
|2020-01-01 00:02:00|7158.82|7158.82|7158.82|
|2020-01-01 00:03:00|7158.82|7158.82|7156.9 |
|2020-01-01 00:04:00|7158.5 |7158.5 |7154.97|
|2020-01-01 00:05:00|7156.52|7159.51|7150.1 |
+-------------------+-------+-------+-------+
only showing top 5 rows
```

Defining Multiple Filters Within a Projection

To define multiple filters we can use binary logic operators to define how our filters are to be combined.

Lets create a projection which only contains entries recorded after `2020-01-01`, **and** whose 'Open' field value is greater than 7000:

```
filter_1 = F.col('Timestamp') > filter_date
filter_2 = F.col('Open') > 7000.0
# We use the '&' symbol to define the logical intersection of our filters.
df.where(filter_1 & filter_2).show(5, False)
```

Which should return

```
+-------------------+-------+-------+-------+-------+-----------+-------------
---+--------------+----+
|Timestamp          |Open   |High   |Low    |Close
|Volume_(BTC)|Volume_(Currency)|Weighted_Price|Year|
+-------------------+-------+-------+-------+-------+-----------+-------------
---+--------------+----+
|2020-01-01 00:01:00|7161.51|7161.51|7155.09|7161.2 |3.7769244   |27047.305
|7161.198      |2020|
|2020-01-01 00:02:00|7158.82|7158.82|7158.82|7158.82|0.02927792  |209.59535
|7158.82       |2020|
|2020-01-01 00:03:00|7158.82|7158.82|7156.9 |7156.9 |0.06581935  |471.1561
|7158.322      |2020|
|2020-01-01 00:04:00|7158.5 |7158.5 |7154.97|7157.2 |0.97138673  |6950.501
|7155.236      |2020|
|2020-01-01 00:05:00|7156.52|7159.51|7150.1 |7158.5 |0.88693166  |6342.851
|7151.454      |2020|
+-------------------+-------+-------+-------+-------+-----------+-------------
---+--------------+----+
only showing top 5 rows
```

It's important to note from the above syntax that we use *binary operators* (`&` for 'and', `|` for 'or', `~` for 'not') to combine filtering conditions, instead of Python's built-in logical operators (`and`, `or`, `not`).

Using the latter operators will raise an error, as the underlying data structures being compared are not boolean values (which is the data type that Python's logical operators expect).

We can also write the above projection compactly by using chaining, while also selecting a subset of the DataFrame's columns:

```
df.select('Timestamp', 'Open', 'High', 'Close')\
  .where((F.col('Timestamp') > filter_date) & (F.col('Open') > 7000))\
  .show(5, False)
```

Which returns

```
+-------------------+-------+-------+-------+
|Timestamp          |Open   |High   |Close  |
+-------------------+-------+-------+-------+
|2020-01-01 00:01:00|7161.51|7161.51|7161.2 |
|2020-01-01 00:02:00|7158.82|7158.82|7158.82|
|2020-01-01 00:03:00|7158.82|7158.82|7156.9 |
|2020-01-01 00:04:00|7158.5 |7158.5 |7157.2 |
|2020-01-01 00:05:00|7156.52|7159.51|7158.5 |
+-------------------+-------+-------+-------+
only showing top 5 rows
```

Note the presence of double brackets in the above statement, which are used to logically separate our filtering conditions that we've defined inline.

Conclusion

Through the effective application of projections and filters within Spark, data can be precisely tailored to meet specific analytical requirements.

▼Key Takeaways ✖

- **Takeaway 1:** Projections, implemented using `select()`, allow for the selection of specific columns, while significantly refining data subsets.
- **Takeaway 2:** Filters, implemented with `filter()` or `where()`, enable the extraction of rows based on defined criteria.
- **Takeaway 3:** We can combine multiple filters using binary logic operators (`&`, `|`, `~`) and chaining `select()` and `where()` methods offers a powerful and concise approach to data manipulation.